Source-Code Evidence for the Real-Time AI Governance Audit Pipeline

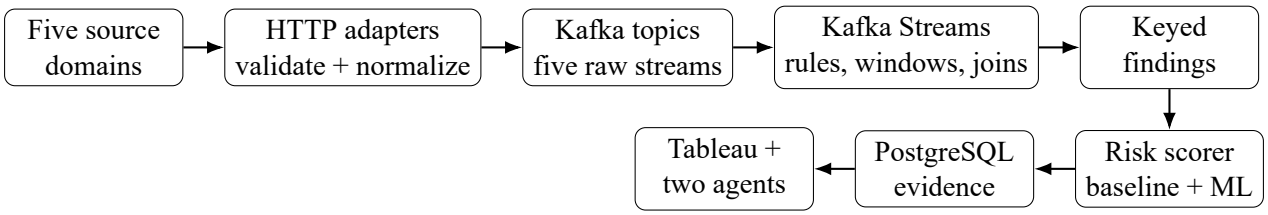
Aditya Ranjan Arindam Gupta Gnanendra Naidu N

November 2026

Abstract

This document records the implemented pipeline using screenshots generated directly from the repository source files. Each screenshot is tied to a source path and represents the corresponding stage described in the project report. The sequence follows the exact flow: producer adapters, Kafka topics and configuration, Kafka Streams processing, governance rules, risk scoring, PostgreSQL evidence, agent investigation and review, evaluation and recovery, and reporting.

Pipeline represented in the report



The screenshots are not mock-ups or reconstructed examples. They are rendered views of source excerpts read from the working repository. The source path is printed in each image header so that every implementation claim can be traced to a file.

1 1. Producer adapters and event normalization

The pipeline starts with five producer domains: Keycloak identity, Kubernetes audit, Airflow/OpenLineage pipeline activity, model registry activity, and LLM gateway activity. The shared mapper layer converts source-specific payloads into the canonical governance-event contract while retaining source identifiers and evidence metadata.



Figure 1: Actual GitHub source page for the producer adapter mapping layer.



Figure 2: Actual GitHub source page for Kafka-compatible broker, topic, and service configuration.

2 2. Kafka topics and stream processing

The normalized events are published to dedicated raw topics. The stream processor consumes those topics using Kafka Streams. The implementation uses event timestamps, state stores, windows, table-table joins, stream-table joins, keyed findings, and JSON serialization. The screenshot below is the actual topology source used to connect those operations.



Figure 3: Actual GitHub source page for the Kafka Streams topology, rules, windows, joins, and finding emission.

3 3. Governance rule catalogue

The rule catalogue is configuration-driven. It associates executable controls with severity and governance-framework metadata. Rule identity is preserved in every finding so that a reviewer can connect a result to the exact configured condition.



Figure 4: Actual GitHub source page for the configured governance rule catalogue.

4 4. Risk scoring and durable evidence

After stream processing, findings are scored and persisted. The risk service keeps the deterministic baseline visible and adds an online Isolation Forest comparison once sufficient history is available. The PostgreSQL sink performs idempotent writes for events, findings, evidence, and scores.

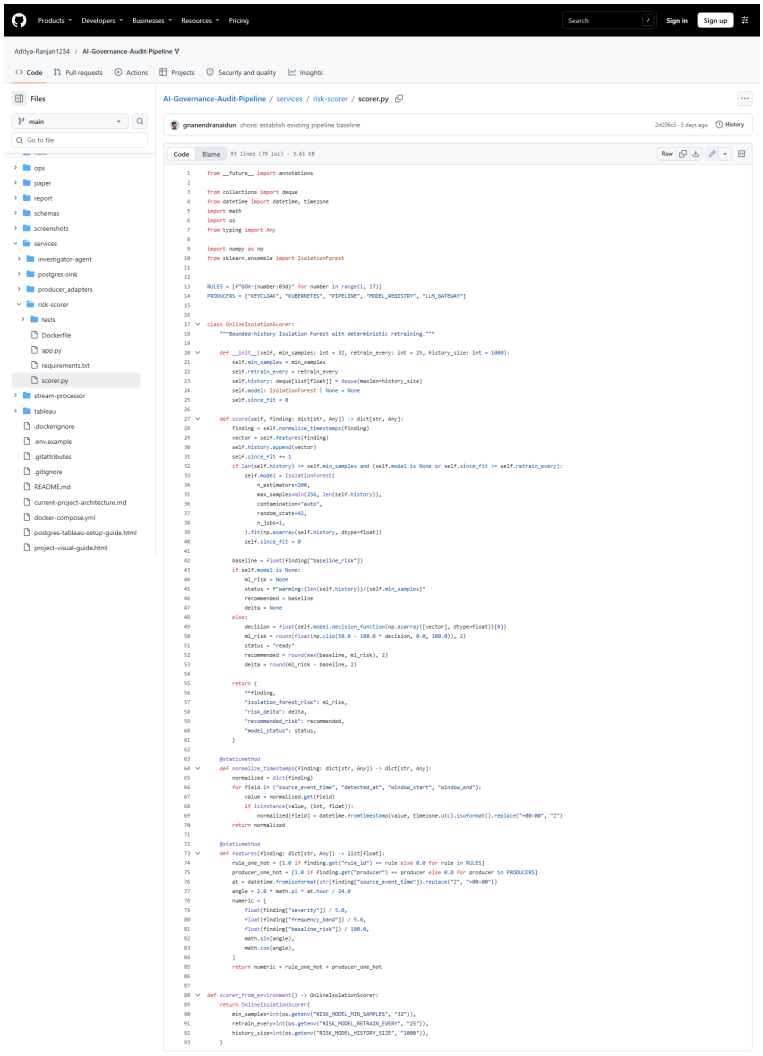


Figure 5: Actual GitHub source page for deterministic baseline and Isolation Forest risk scoring.



Figure 6: Actual GitHub source page for durable, idempotent PostgreSQL evidence persistence.

5 5. Investigator and Evidence Reviewer agents

The agents run after durable evidence exists. The Investigator retrieves eligible evidence and writes a structured report. The Evidence Reviewer checks citations and completeness, may request a bounded correction, and can terminate with human review. Neither agent is an infrastructure-remediation authority.



Figure 7: Actual GitHub source page for Amazon Bedrock Investigator report generation.



Figure 8: Actual GitHub source page for Evidence Reviewer validation, bounded revision, and terminal states.

6 6. Evaluation and recovery evidence

The repository evaluation uses labelled source-shaped events and expected findings. The recorded demonstration contains 72 events and 48 expected findings across 16 rules. Baseline, processor-replacement, broker-restart, adapter, consumer-tail, and agent-review checks are kept as repository artefacts.

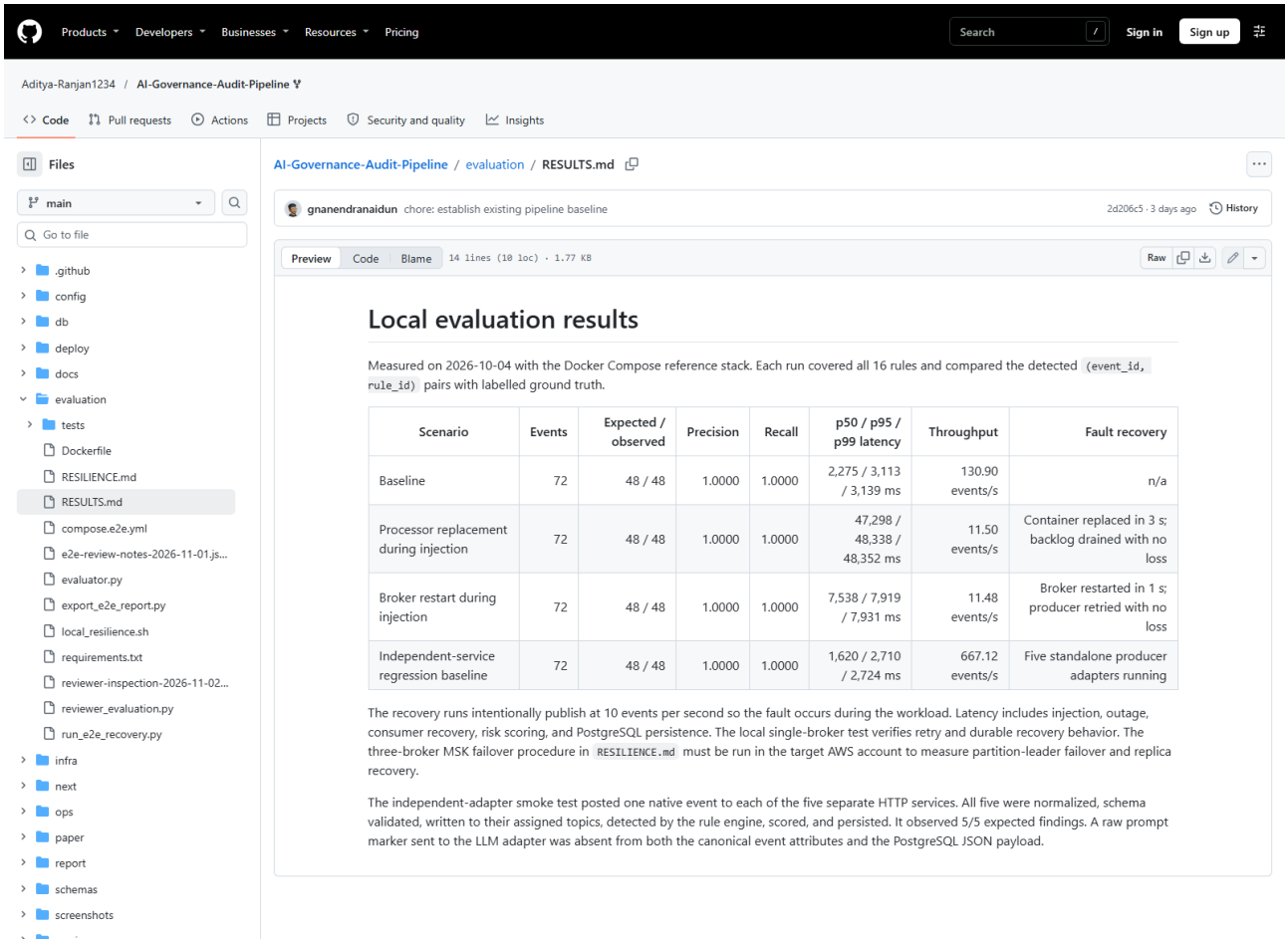


Figure 9: Actual GitHub source page for recorded evaluation results.

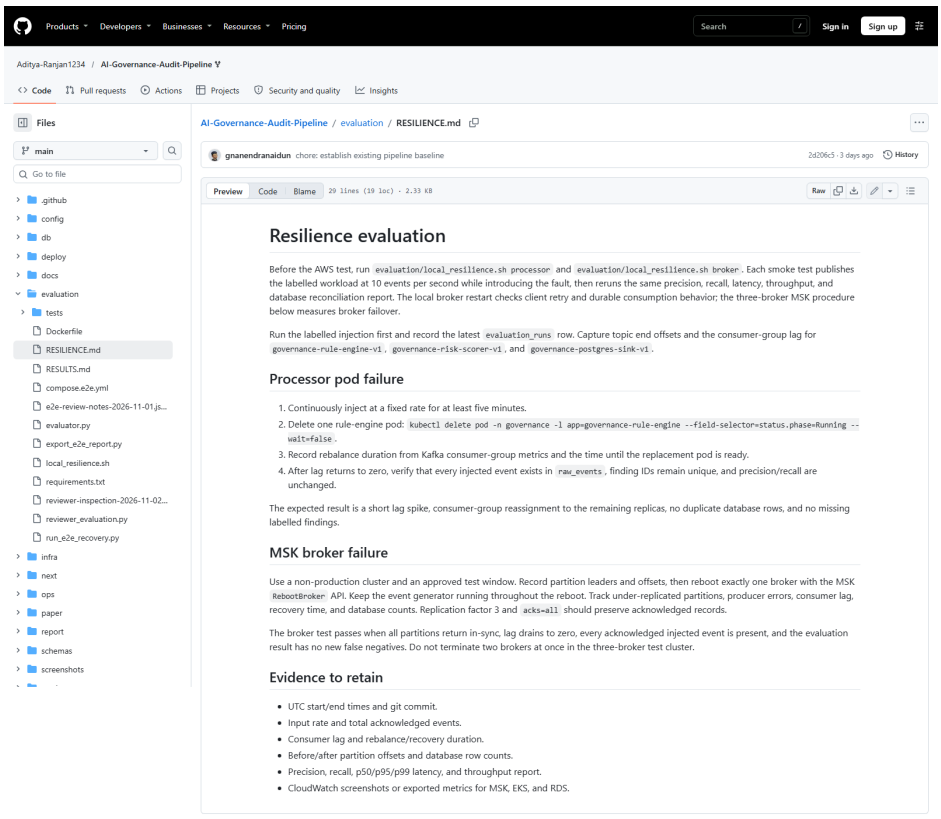


Figure 10: Actual GitHub source page for restart, replay, and resilience verification.

7 7. Reporting output

The final stage exposes durable evidence through reporting views. The dashboard consumes stored findings, risk scores, framework associations, latency observations, and incident details. It is downstream of the detection and persistence path and does not replace the evidence records.

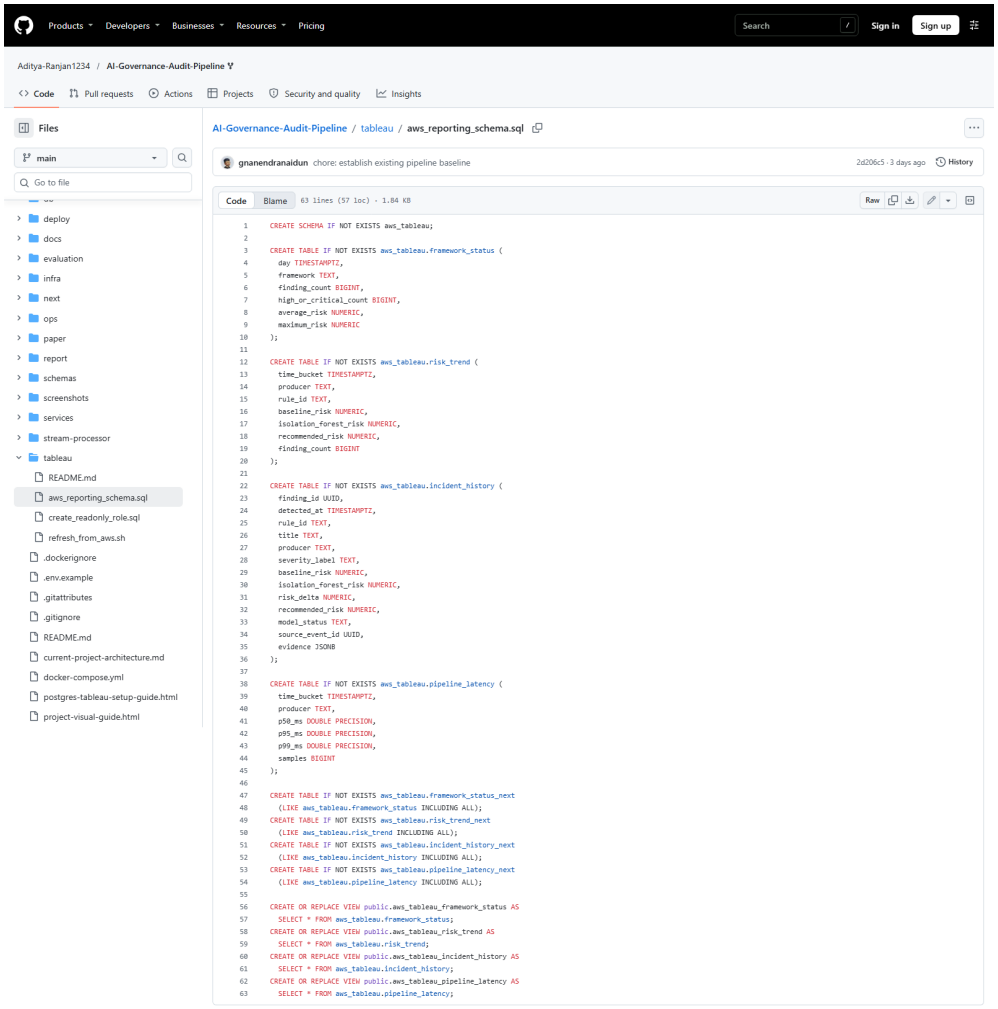


Figure 11: Actual GitHub source page for the reporting schema exposing persisted governance evidence.

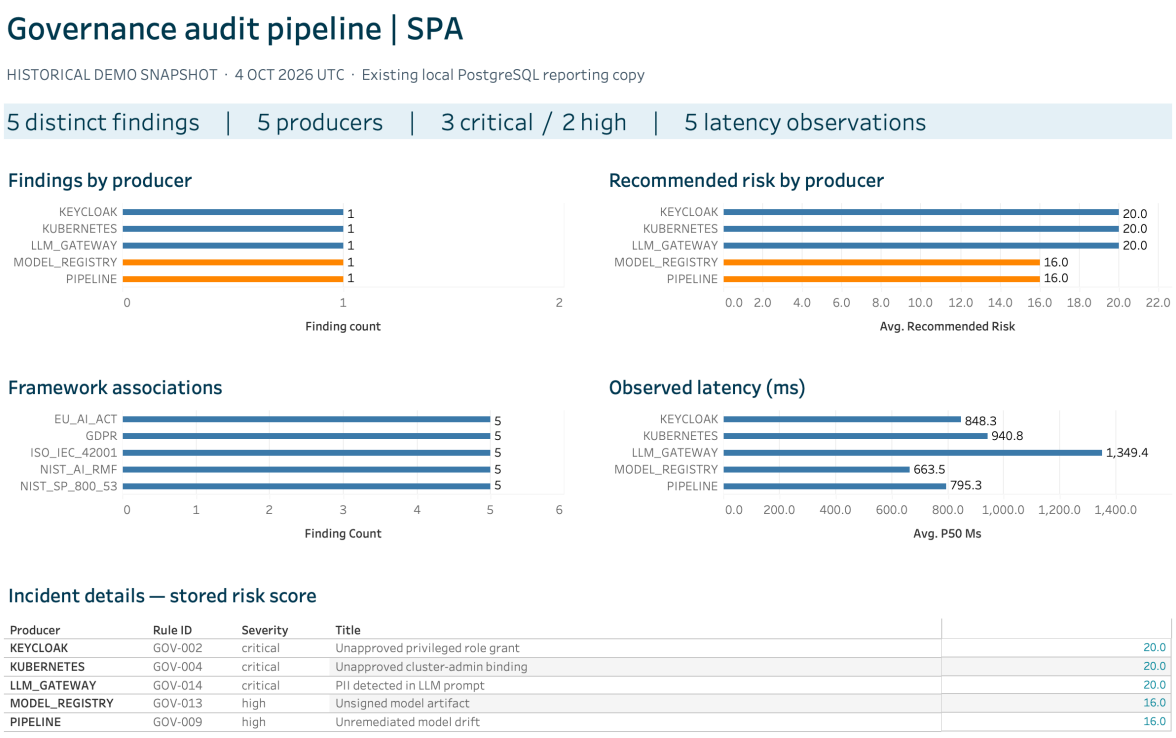


Figure 12: Repository-provided Tableau dashboard evidence for the reporting stage.

Traceability summary

Pipeline stage	Source evidence included
Producer adapters	Mapper and adapter configuration source
Kafka boundary	Docker Compose broker/topic configuration
Stream processor	Java Kafka Streams topology
Governance controls	YAML rule catalogue
Risk and storage	Python scorer and PostgreSQL sink
Agent review	Investigator and Reviewer source
Evaluation	Results and resilience artefacts
Reporting	Tableau reporting schema and dashboard